Rename `system_context_replaceability` namespace

Document #:	P4031R1 [Latest] [Status]
Date:	2026-03-24
Project:	Programming Language C++
Audience:	LWG

Reply-to:	Ruslan Arutyunyan <ruslan.arutyunyan@intel.com>

Contents

§ Abstract

This paper proposes to rename `system_context_replaceability` namespace.

§ 1 Motivation

There was one attempt to rename `system_context_replaceability`. Unfortunately, it didn't go well because likely the proposed alternatives for the names were not received well. [P3804R1] proposed several options favoring `replacement` and `replacement_functions`. As people pointed out, those names are probably also not great because they don't give any sense of what developers try to replace. For example `std::execution::replacement::receiver_proxy` doesn't help to understand what we're actually replacing. The motivation to recommend either `replacement` or `replacement_functions` came because `system_context_replaceability` felt too long for a namespace name. However, people don't seem to care about the length of the name. The poll below taken in Kona meeting (2025) shows that:

POLL: Change namespace `system_context_replacability` to `replacement` as proposed in P3804R0 Iterating on `parallel_scheduler`.

SF	F	N	A	SA
2	1	6	5	1

What is also important to note is that the poll was only about changing the current status quo to `replacement`. We have not voted for other options in [P3804R1]. Nevertheless, in my opinion we have to rename `system_context_replaceability` to something else because what we should care about is the consistency of C++ standard (whenever possible). Adding `system_context_replaceability` namespace name to C++26 will cause a lot of confusion because `system_context` does not even exist in the current working draft. Thus, users would not understand what they are actually replacing. What we have in the current working draft is actually named `parallel_scheduler` and the replaceability feature was developed specifically for it. I understand the argument that suggested `replacement` option perhaps does not give us what we want but it doesn't eliminate the fact that we should rename `system_context_replaceability` to avoid even bigger confusion because `std::execution::system_context_replaceability::receiver_proxy` with no `system_context` presence anywhere in the C++ working draft is no better.

Based on the above either of `parallel_scheduler_replaceability` or `parallel_scheduler_replacement` name should be good enough because people seem to not be bothered by the name length and also because `parallel_scheduler` is exactly the name we have for the scheduler itself in the current working draft. Leaving the status quo would be a disaster, in my opinion.

§ 2 Proposal

Option 1: Change `std::execution::system_context_replaceability` namespace name to `std::execution::parallel_scheduler_replaceability`.

Option 2: Change `std::execution::system_context_replaceability` namespace name to `std::execution::parallel_scheduler_replacement`.

Modify the formal wording in accordance with the winning option (if any wins compared to the status quo).

I like the second option slightly better because *replaceability* word means the feature to me and *replacement* sounds more like a set of the actual API. But I can live with any of those.

§ 3 Outcome

In Croydon 2026, LEWG voted in favor of `system_context_replaceability` renaming and the chosen name is `std::execution::parallel_scheduler_replacement`, which is Option 2 in Proposal section.

§ 3.1 Polls

POLL: We want to rename `system_context_replaceability` to either `parallel_scheduler_replaceability` or `parallel_scheduler_replacement`

SF	F	N	A	SA
6	8	0	0	0

Outcome: Consensus in favor

POLL: We want to rename `system_context_replaceability` to either `parallel_scheduler_replaceability` or `parallel_scheduler_replacement`

Option1: `parallel_scheduler_replaceability`

Option2: `parallel_scheduler_replacement`

S1	W1	N	W2	S2
0	2	4	6	1

POLL: Apply the change in "P4031R0" with the name: `parallel_scheduler_replacement` to address "US 205-320 33.4 [execution.syn] Rename `system_context_replaceability` namespace P4031." and send to LWG.

Outcome: No objection to unanimous consent

§ 4 Wording

[Editor's note: The diff is against the current C++ Working Draft. There could be merge conflicts because more papers that are near `system_context_replaceability` on the

way. Eventually `system_context_replaceability` should be renamed to `parallel_scheduler_replacement` all over the places as well as sysctxrepl stable name should be renamed to parschedrepl]

§ 4.1 Modify [execution.syn]

```
// [exec.par.scheduler], parallel scheduler
class parallel_scheduler;
parallel_scheduler get_parallel_scheduler();

-// [exec.sysctxrepl], namespace system_context_replaceability
-namespace system_context_replaceability {
+// [exec.parschedrepl], namespace parallel_scheduler_replacement
+namespace parallel_scheduler_replacement {
  struct receiver_proxy;
  struct bulk_item_receiver_proxy;
  struct parallel_scheduler_backend;

                                    shared_ptr<parallel_scheduler_backend>
        query_parallel_scheduler_backend();
}
```

§ 4.2 Modify [exec.par.scheduler]

6 *A preallocated backend storage for a proxy* `r` is an object `s` of type `span<byte>` such that the range `s` remains valid and may be overwritten until one of `set_value`, `set_error`, or `set_stopped` is called on `r`.

[*Note:* The storage referenced by `s` can be used as temporary storage for operations launched via calls to `parallel_scheduler_backend`. — end note — *end note*]

7 *A bulk chunked proxy for `rcvr` with callable `f` and arguments `args`* is a proxy `r` for `rcvr` with base ~~`system_context_replaceability`~~`parallel_scheduler_replacement::bulk_item_receiver_proxy` such that `r.execute(i, j)` for indices `i` and `j` has effects equivalent to `f(i, j, args...)`.

8 *A bulk unchunked proxy for `rcvr` with callable `f` and arguments `args`* is a proxy `r` for `rcvr` with base ~~`system_context_replaceability`~~`parallel_scheduler_replacement::bulk_item_receiver_proxy` such that `r.execute(i, i + 1)` for index `i` has effects equivalent to `f(i, args...)`.

9 Let `b` be `BACKEND-OF(sch)`, let `sndr` be the object returned by `schedule(sch)`, and let `rcvr` be a receiver. If `rcvr` is connected to `sndr` and the resulting operation state is started, then:

(9.1) — If `sndr` completes successfully, then `b.schedule(r, s)` is called, where

(9.1.1) — `r` is a proxy for `rcvr` with base ~~`system_context_replaceability`~~`parallel_scheduler_replacement::receiver_proxy` and

(9.1.2) — `s` is a preallocated backend storage for `r`.

(9.2) — All other completion operations are forwarded unchanged.

[…]

11 `parallel_scheduler` provides a customized implementation of the `bulk_unchunked` algorithm ([exec.bulk]). If a receiver `rcvr` is connected to the sender returned by `bulk_unchunked(sndr, pol, shape, f)` and the resulting operation state is started, then:

(11.1) — If `sndr` completes with values `vals`, let `args` be a pack of lvalue subexpressions designating `vals`, then `b.schedule_bulk_unchunked(shape, r, s)` is called, where

(11.1.1) — `r` is a bulk unchunked proxy for `rcvr` with callable `f` and arguments `args` and

(11.1.2) — `s` is a preallocated backend storage for `r`.

(11.2) — All other completion operations are forwarded unchanged.

```
parallel_scheduler get_parallel_scheduler();
```

12 *Effects*: Let `eb` be the result of ~~`system_context_replaceability`~~`parallel_scheduler_replacement::query_parallel_scheduler_backend()`. If `eb == nullptr` is `true`, calls `terminate` ([except.terminate]). Otherwise, returns a `parallel_scheduler` object associated with `eb`.

§ 4.3 Modify Namespace ~~`system_context_replaceability`~~`parallel_scheduler_replacement` [exec.~~`sysctxrepl`~~`parschedrepl`]

§ 4.4 Modify [exec.~~`sysctxrepl`~~`parschedrepl`.general]

1 Facilities in the ~~`system_context_replaceability`~~`parallel_scheduler_replacement` namespace allow users to replace the default implementation of `parallel_scheduler`.

§ 4.5 Modify Receiver proxies [exec.~~`sysctxrepl`~~`parschedrepl`.recvproxy]

```
namespace      std::execution::system_context_replaceabilityparallel_sched-
        uler_replacement {
  struct receiver_proxy {
    virtual ~receiver_proxy() = default;

  protected:
    virtual bool query-env(unspecified) noexcept = 0;   // exposition only

  public:
    virtual void set_value() noexcept = 0;
    virtual void set_error(exception_ptr) noexcept = 0;
    virtual void set_stopped() noexcept = 0;

    template<class P, class-type Query>
```

```
      optional<P> try_query(Query q) noexcept;
  };

  struct bulk_item_receiver_proxy : receiver_proxy {
    virtual void execute(size_t, size_t) noexcept = 0;
  };
}
```

§ 4.6 Modify `query_parallel_scheduler_backend` [exec.~~sysctxrepl~~parschedrepl.query]

§ 4.7 Modify Class `parallel_scheduler_backend` [exec.~~sysctxrepl~~parschedrepl.psb]

```
namespace      std::execution::~~system_context_replaceability~~parallel_sched-
        uler_replacement {
  struct parallel_scheduler_backend {
    virtual ~parallel_scheduler_backend() = default;

    virtual void schedule(receiver_proxy&, span<byte>) noexcept = 0;
    virtual void schedule_bulk_chunked(size_t, bulk_item_receiver_proxy&,
                                       span<byte>) noexcept = 0;
                     virtual      void      schedule_bulk_unchunked(size_t,
        bulk_item_receiver_proxy&,
                                         span<byte>) noexcept = 0;
  };
}
```

§ 5 Revision History

§ 5.1 R0 => R1

— Apply the LEWG decision from Croydon, 2026 to rename `system_context_replaceability` to `parallel_scheduler_replacement`

— Add wording

§ 6 References

[P3804R1] Lucian Radu Teodorescu, Ruslan Arutyunyan. 2025-12-15. Iterating on parallel_scheduler.
https://wg21.link/p3804r1